to fit their full-precision versions into memory. In terms of memory footprint reduction, as shown in Figure 12, quantization significantly reduces memory usage, with reductions by factors between 1.7 and 3 across all models, devices, and quantization schemes. However, as previously discussed, *Q4_0* quantization exhibits a higher memory footprint for CPU inference but a lower footprint for GPU inference compared to *Q4_K_M* quantization. Analyzing the effects on token throughput and energy consumption, *Q4_K_M* quantization negatively impacted both token throughput and energy consumption in the prefill phase, with varying intensity depending on the model and device. Throughput reductions ranged between factors of 1.09 and 4.25, while energy consumption increased by factors between 1.04 and 7.57. Conversely, in the generation phase, *Q4_K_M* quantization led to performance improvements, where token throughput increased by factors between 1.17 and 2.76, and energy consumption decreased by factors ranging from 1.02 to 2.17. When it comes to outliers, Gemma 2 2B exhibited unexpected improvements in both prefill throughput and energy consumption for GPU inference. Flan T5 Large showed degraded performance in generation throughput and energy consumption on the Orin CPU. In contrast, *Q4_0* quantization yielded consistent improvements in both throughput and energy consumption across both phases for CPU inference, while varying results were observed for GPU inference depending on the model. For Gemma 2 2B prefill throughput improved significantly by a factor of 7.18 and energy consumption per token dropped by a factor of 6.33. For other models prefill performance either dropped or stayed approximately constant compared to unquantized models. However, for Qwen 2 1.5B and Flan T5 Large energy consumption dropped by factors 1.24 and 1.12 respectively, while for Llama 3.2 1B and Qwen 2 0.5B energy consumption increased by factors of 1.52 and 3.36. The improved performance in the generation phase can be attributed to the memory-bound nature of this phase. Since quantized model weights require lower precision, a greater portion of them fits into cache, leading to fewer cache misses and ultimately faster token generation.

Main Takeaways

- In terms of perplexity, *Q4_K_M* quantization performed better than *Q4_0* quantization for most models, especially for the smallest evaluated models. Overall, perplexity increases was especially pronounced for the smallest models.
- *Q4_0* quantization improved memory footprint, token throughput, and energy consumption over full-precision models, while *Q4_K_M* quantization resulted in inferior throughput and energy consumption in the prefill phase.

6 Analysis and Trade-Offs Discussion

Optimizing LM inference at the edge requires balancing multiple constraints, including computational performance, energy efficiency, and cost. In this section, we leverage our empirical measurements to analyze key trade-offs, assess its practical usability in real-world deployments, and evaluate the economic feasibility of LM inference on edge devices.

6.1 Throughput-Energy Trade-Off

On the Orin CPU, the fastest configuration was either also the most energy-efficient or, at worst, only marginally less efficient than the optimal configuration. However, no single configuration simultaneously maximized performance and energy efficiency on the RPi 5 or the Orin GPU. To determine the optimal

trade-off between token throughput and energy consumption, we define the metric action per token S_{t} as the ratio between the energy E consumed during an inference phase and the token throughput t:

$$S_{\mathrm{t}} = \frac{E}{t}$$

This metric represents the amount of action required to process a token during the prefill phase or generate a token in the generation phase. A lower action per token value indicates higher efficiency and is therefore considered preferable in our analysis. The term *action* is used because the resulting unit, $\frac{\text{joule-seconds}}{\text{token}}$, corresponds to the physical unit of action in physics. Additionally, the symbol S was chosen as it is commonly used to denote action in physics.

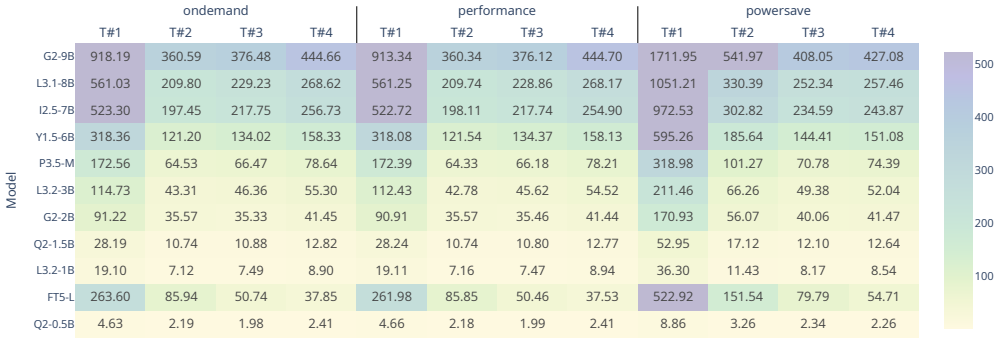
(a) (b)

Fig. 13. Comparison of generation action per token on RPi 5 for *Q4_K_M* (a) and *Q4_0* (b).

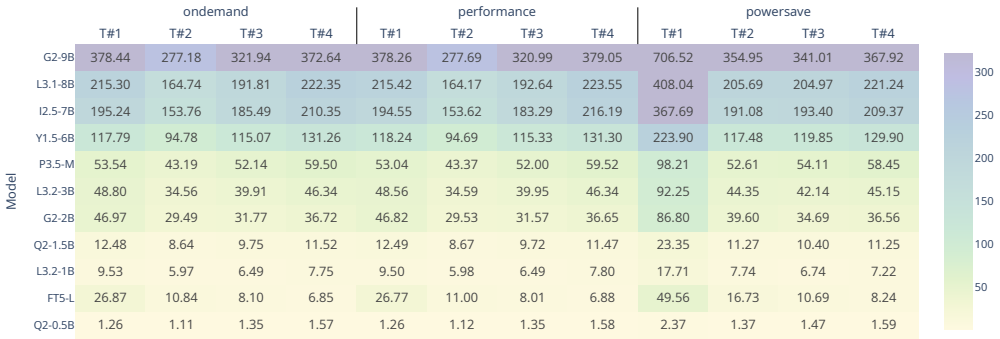
When calculating S_t for the prefill phase, Figure 13 show that the *ondemand* and *performance* governors achieved a 36–38% lower action per token than the *powersave* governor for *Q4_K_M* quantization. The same trend was observed for *Q4_0* quantization for which the *ondemand* and *performance* governors resulted in 15-31% lower action per token. Only results for 4 threads are reported, as this configuration provided the best balance between energy consumption and token throughput across all models.

When analyzing the generation phase results in Figure 14, it is evident that for *Q4_K_M* quantization, the *ondemand* and *performance* governors also yielded the lowest action per token. For *Q4_0* quantization, for all models 2 threads were optimal with the only exception being Flan T5 Large for which 4 threads were optimal. *Q4_K_M* quantization showed similar results with the only differences found for the models Gemma 2 2B and Qwen 2 0.5B, for which 3 threads minimized action per token. Overall, these results suggest that for decoder-only models, the *ondemand* and *performance* governors perform with 2 threads perform best for most models, with some exceptions showing slightly better efficiancy with 3 threads.

On the Orin GPU, the *15W* mode resulted in the lowest energy consumption across all models, while the *MAX* mode provided the highest inference speeds at the cost of increased energy usage. When using action per token as the decisive metric, it becomes evident that, for most smaller models, the *MAX* mode offered the best energy-throughput trade-off in the prefill phase for both quantization schemes, while for the largest

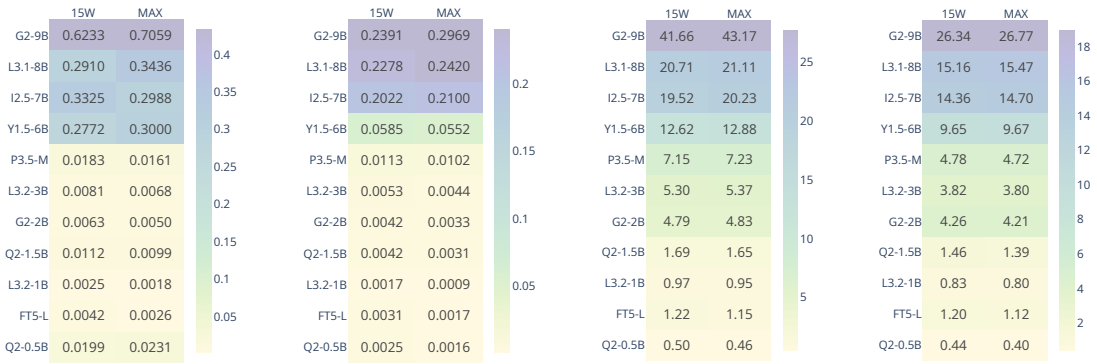


(a)



(b)

Fig. 14. Comparison of generation action per token on RPi 5 for *Q4_K_M* (a) and *Q4_0* (b).



(a) Prefill, *Q4_K_M* (b) Prefill, *Q4_0* (c) Generation, *Q4_K_M* (d) Generation, *Q4_0*

Fig. 15. Action per token for prefill and generation phase for all power modes on Orin GPU